



SOFTWARE REQUIREMENTS SPECIFICATION

Android 360° Panorama Capture & Virtual Tour App

A native Android application enabling real estate professionals to capture, stitch, and publish immersive 360° virtual property tours — integrated seamlessly with the Ethred Backend API.

DOCUMENT VERSION v2.0 (Final)	DATE July 2026	PLATFORM Android 7.0+
BACKEND Ethred API v1	STATUS Approved	CLASSIFICATION Internal

Table of Contents

1. Introduction & Executive Summary	3
1.1 Purpose of This Document	3
1.2 Project Overview	3
1.3 Backend Integration Boundary Analysis	4
1.4 Definitions, Acronyms & Glossary	4
2. System Architecture	5
2.1 High-Level Architecture Diagram	5
2.2 Android App Module Architecture	5
2.3 On-Device Stitching Pipeline	6
2.4 Technology Stack	6
3. Data Schema & Backend Integration	7
3.1 Existing Prisma Schema (Preserved)	7
3.2 Local Room Database Schema	7
3.3 Virtual Tour Manifest JSON Schema	7
3.4 API Integration Contracts	8
4. Functional Requirements	9
4.1 Authentication & Authorization (FR-AUTH)	9
4.2 Gyroscope-Guided Frame Capture (FR-CAP)	9
4.3 On-Device Stitching Engine (FR-STITCH)	11
4.4 Gap Inpainting & Post-Processing (FR-FILL)	12
4.5 Multi-Room Tour Builder (FR-TOUR)	12
4.6 Backend Upload & Sync (FR-SYNC)	13
4.7 Local Preview Viewer (FR-VIEW)	14
5. Non-Functional Requirements	14
6. Use Case Scenarios	16
7. Screen & UI/UX Specification	18
8. Error Handling & Edge Cases	19
9. Security Requirements	20
10. Testing Strategy	20

11. Developer Implementation Prompt 21

1 Introduction & Executive Summary

1.1 Purpose of This Document

This Software Requirements Specification (SRS) defines the complete, authoritative, and developer-ready requirements for the **Android 360° Panorama Capture & Virtual Tour Application**. It is intended to be used directly by Android developers, QA engineers, and frontend integration engineers building the Pannellum.js-based viewer without any ambiguity.

This document covers all functional requirements, non-functional constraints, data schema integration contracts, use case scenarios, security requirements, testing strategy, and a ready-to-use prompt for AI-assisted development. Every requirement is explicitly aligned with the **existing Ethred Backend API** to guarantee zero breaking changes.

1.2 Project Overview

The **Ethred platform** is a real estate marketplace operating in Ethiopia. Property agents and sellers currently upload flat 2D images and videos of properties. The objective of this project is to allow agents to capture **professional-quality 360° equirectangular panoramas** of property rooms using nothing more than an ordinary Android smartphone with a hardware gyroscope, and to publish those panoramas as interactive virtual tours viewable on the Ethred web platform — exactly like those seen on Redfin or Matterport.



1.3 Backend Integration Boundary Analysis & Conflict Resolution

The original SRS concept conflicted with the existing backend in the following ways. This section documents what changed and why:

Issue Identified	Original Conflict	Resolution in This SRS
Server-Side Stitching	Required a dedicated GPU compute server and new database tables for processing jobs	Stitching runs entirely on-device using native OpenCV C++ via Android NDK. Zero server cost.
Custom Media Schema	Assumed a new <code>panorama_session</code> table and media type	Uses the existing <code>PropertyMedia</code> table with <code>media_category: IMAGE</code> for panoramas and <code>DOCUMENT</code> for manifest files.
New API Endpoints	Proposed new REST endpoints for 3D session management	Uses only existing endpoints: <code>POST /api/v1/auth/login</code> , <code>POST /api/v1/properties</code> , and the media upload endpoint.
Proprietary 3D File Formats	Required .matterport or .ptx format support	Output is standard 2:1 Equirectangular JPEG with GPano EXIF XMP metadata — universally supported by Pannellum.js without any backend parsing.
Tour Linking	Multi-room linking required relational DB tables for hotspot positions	A lightweight <code>tour_manifest.json</code> is generated by the app and uploaded as a <code>DOCUMENT</code> media asset. Frontend Pannellum.js reads this manifest to build the tour.

1.4 Definitions, Acronyms & Glossary

Term	Definition
Equirectangular	A 2:1 aspect ratio spherical panorama projection where the full 360°×180° field of view is mapped onto a flat rectangular image. Industry standard for interactive panoramic viewers.
OpenCV	Open Source Computer Vision Library. An Apache 2.0 licensed, royalty-free C++ library for image processing, feature detection, and panoramic stitching. Used via Android NDK (Native Development Kit).
CameraX	Google's Jetpack library providing a consistent, hardware-optimized camera API across Android devices from API 21+.
Gyroscope / IMU	Inertial Measurement Unit. The hardware sensor providing real-time 3-axis angular rotation data used to track phone orientation during capture.
Zenith / Nadir	The top pole (sky/ceiling) and bottom pole (floor/feet) of a sphere respectively. In handheld capture these areas are missing and must be filled via inpainting.
Inpainting	An image restoration algorithm (cv::inpaint) that fills masked/missing pixel regions by propagating texture and color from surrounding pixels.
Pannellum.js	A lightweight, WebGL-based open-source JavaScript panorama viewer that renders equirectangular images interactively in a browser.
AKAZE	Accelerated-KAZE. A robust feature descriptor algorithm used for keypoint detection and matching across overlapping images during stitching. Superior to SIFT/ORB for panoramic stitching accuracy.
Homography	A 3×3 projection matrix describing the geometric relationship between two camera views, computed from matched feature keypoints. Used to warp image planes onto a sphere.
WorkManager	Android Jetpack library that manages guaranteed background task execution, including network-resilient file uploads.
Room DB	Android Jetpack local SQLite abstraction layer used to persist capture sessions, queue items, and uploaded metadata locally on the device.
JNI	Java Native Interface. The bridge between Kotlin/Java Android code and native C++ OpenCV library functions.
GPano XMP	Google's Photo Sphere XMP metadata standard embedded in JPEG files that signals to browsers and viewers that the image is a full 360° panorama.

2 System Architecture

2.1 High-Level Architecture Diagram



2.2 Android App Module Architecture (MVVM + Clean Architecture)

Layer	Module / Package	Responsibility
Presentation	<code>ui.capture</code> , <code>ui.tour</code> , <code>ui.viewer</code>	Jetpack Compose screens, ViewModels, UI state flows
Domain	<code>domain.usecase</code>	CaptureSessionUseCase, StitchPanoramaUseCase, UploadTourUseCase — pure business logic
Data	<code>data.repository</code>	CaptureSessionRepository, PropertyRepository — coordinates local Room DB and remote API calls
Native/NDK	<code>jni/</code> C++ sources	OpenCV stitching pipeline, inpainting, EXIF writer — exposed via JNI bridge
Local Persistence	<code>data.local.db</code>	Room DB entities: CaptureSessionEntity, FrameEntity, HotspotEntity, UploadQueueEntity
Network	<code>data.remote</code>	Retrofit2 API client, AuthInterceptor (JWT), multipart upload service
Background	<code>worker</code>	Android WorkManager workers: StitchWorker, UploadWorker, ManifestWorker

2.3 On-Device Stitching Pipeline (Step-by-Step)

- 1. Frame Input:** 24–36 raw JPEG frames (3264×2448 px each) passed to native C++ via JNI as byte arrays.
- 2. Decode & Resize:** OpenCV decodes each frame into a `cv::Mat` and downsamples to 1920×1440 px for feature detection (full resolution used for final blend).
- 3. AKAZE Feature Detection:** Per-frame keypoints and binary descriptors extracted using `cv::AKAZE::create()`.
- 4. Feature Matching & Filtering:** FLANN-based matcher with ratio test (Lowe's ratio 0.75) identifies valid correspondences. Homography validated via RANSAC.
- 5. Gyro-Assisted Camera Rotation Estimation:** Initial rotation estimates seeded from logged Rotation Vector sensor data, refined by homography decomposition for accuracy.
- 6. Spherical Warping:** Each frame warped onto a sphere using `cv::detail::SphericalWarper` with focal length estimated from EXIF FOV data or manual calibration lookup table per device model.
- 7. Seam Finding:** Optimal seams between overlapping warp regions computed with `cv::detail::GraphCutSeamFinder` for the most visually seamless transitions.
- 8. Exposure Compensation:** `cv::detail::BlocksGainCompensator` equalizes brightness/color temperature across all frames before blending.
- 9. Multi-Band Blending:** `cv::detail::MultiBandBlender` with 5 pyramid levels blends seam regions to eliminate ghost artifacts.
- 10. Equirectangular Crop & Scale:** Output cropped to 2:1 aspect ratio (target 8192×4096 px for high-end, 4096×2048 for standard). Missing pole regions filled with mask for inpainting pass.
- 11. Inpainting Pass:** `cv::inpaint()` with Navier-Stokes algorithm fills zenith and nadir regions. Nadir optionally receives agency logo cap overlay.
- 12. EXIF XMP Write:** GPano metadata tags written to output JPEG: `ProjectionType=equirectangular`, `UsePanoramaViewer=true`, `FullPanoWidthPixels`, `FullPanoHeightPixels`.
- 13. Output:** Final equirectangular JPEG saved to local `Room360/sessions/{session_id}/` directory.

2.4 Technology Stack

Component	Technology	Version	License
Language	Kotlin	1.9+	Apache 2.0
UI Framework	Jetpack Compose	1.5+	Apache 2.0
Camera	CameraX	1.3+	Apache 2.0
Image Processing	OpenCV Android SDK	4.8+	Apache 2.0 (Free)
Native Bridge	Android NDK + JNI	r25c+	BSD / Free
Local Database	Room + SQLite	2.6+	Apache 2.0
Network	Retrofit2 + OkHttp3	2.9 / 4.12	Apache 2.0
Async	Kotlin Coroutines + Flow	1.7+	Apache 2.0
Background Jobs	WorkManager	2.9+	Apache 2.0
DI Framework	Hilt (Dagger2)	2.48+	Apache 2.0
In-App Preview	SceneView (Three.js WebView)	Latest	MIT
JSON	Kotlinx Serialization / Gson	1.6+	Apache 2.0
Image Load	Coil	2.5+	Apache 2.0

✅ **All Free & Open Source.** Every library in the technology stack is Apache 2.0 or MIT licensed with no per-use fees, API rate limits, or subscription requirements. The total additional server infrastructure cost from this feature is \$0.00.

3 Data Schema & Backend Integration

3.1 Existing Prisma Backend Schema (100% Preserved — DO NOT MODIFY)

```
// Existing backend schema — no changes required or permitted
model PropertyMedia {
  id          String    @id @default(uuid())
  property_id String
  file_url    String    // Stores 360° image URL or tour manifest URL
  media_category MediaType // IMAGE | VIDEO | DOCUMENT
  sort_order  Int
}

// MediaType Enum (existing)
enum MediaType {
  IMAGE    // ← Used for equirectangular 360° panorama JPEGs
  VIDEO
  DOCUMENT // ← Used for tour_manifest.json multi-room linking file
}
```

⚠ **Strict Boundary Rule.** The Android application MUST NOT trigger any schema migrations, new table creation, or enum value additions on the existing database. All 360° media is stored using existing field conventions only.

3.2 Android Local Room Database Schema

The Android app maintains its own local SQLite database (via Room) for managing capture sessions offline and queuing uploads:

```

-- Table: capture_sessions
CREATE TABLE capture_sessions (
  id          TEXT PRIMARY KEY,    -- UUID v4
  property_id TEXT NOT NULL,       -- Foreign key to ethred property
  room_name   TEXT NOT NULL,       -- e.g. "Living Room"
  status      TEXT NOT NULL,       -- CAPTURING | STITCHING | DONE | UPLOADING | UPLOADED | FAILED
  frame_count INTEGER DEFAULT 0,
  output_path TEXT,                -- Local file path of finished panorama JPEG
  created_at  INTEGER,             -- Unix epoch ms
  uploaded_at INTEGER              -- Null until uploaded
);

-- Table: captured_frames
CREATE TABLE captured_frames (
  id          TEXT PRIMARY KEY,
  session_id  TEXT NOT NULL,
  file_path   TEXT NOT NULL,       -- Local path to raw JPEG frame
  yaw_deg     REAL NOT NULL,       -- Gyro yaw at moment of capture
  pitch_deg   REAL NOT NULL,
  roll_deg    REAL NOT NULL,
  captured_at INTEGER
);

-- Table: hotspots
CREATE TABLE hotspots (
  id          TEXT PRIMARY KEY,
  from_session_id TEXT NOT NULL,   -- Scene A
  to_session_id TEXT NOT NULL,     -- Scene B (target)
  pitch       REAL NOT NULL,       -- Pannellum pitch coordinate
  yaw         REAL NOT NULL,       -- Pannellum yaw coordinate
  label       TEXT                 -- e.g. "Go to Kitchen"
);

-- Table: upload_queue
CREATE TABLE upload_queue (
  id          TEXT PRIMARY KEY,
  session_id  TEXT NOT NULL,
  property_id TEXT NOT NULL,
  file_url    TEXT,                -- Populated after successful upload
  media_category TEXT NOT NULL,    -- "IMAGE" or "DOCUMENT"
  sort_order  INTEGER DEFAULT 0,
  retry_count INTEGER DEFAULT 0,
  status      TEXT DEFAULT 'PENDING' -- PENDING | IN_PROGRESS | DONE | FAILED
);

```

3.3 Virtual Tour Manifest JSON Schema

When multiple 360° rooms are linked, the Android app generates a `tour_manifest.json` file that the web frontend reads to construct the interactive multi-scene Pannellum.js tour:

```

{
  "version": "1.0",
  "property_id": "8f2b3e41-0000-4000-a000-123456789abc",
  "default_scene": "scene_living_room",
  "created_at": "2026-07-23T13:49:00Z",
  "scenes": [
    {
      "id": "scene_living_room",
      "name": "Living Room",
      "file_url": "https://ethred-backend.onrender.com/uploads/living_room_360.jpg",
      "thumbnail_url": "https://ethred-backend.onrender.com/uploads/living_room_thumb.jpg",
      "pitch_offset": 0,
      "yaw_offset": 0,
      "hotspots": [
        {
          "id": "hs_lr_to_kitchen",
          "pitch": -5.2,
          "yaw": 112.5,
          "text": "Go to Kitchen",
          "target_scene_id": "scene_kitchen"
        }
      ]
    },
    {
      "id": "scene_kitchen",
      "name": "Kitchen",
      "file_url": "https://ethred-backend.onrender.com/uploads/kitchen_360.jpg",
      "thumbnail_url": "https://ethred-backend.onrender.com/uploads/kitchen_thumb.jpg",
      "pitch_offset": 0,
      "yaw_offset": -180,
      "hotspots": [
        {
          "id": "hs_kitchen_to_lr",
          "pitch": -2.0,
          "yaw": -75.0,
          "text": "Return to Living Room",
          "target_scene_id": "scene_living_room"
        }
      ]
    }
  ]
}

```

3.4 API Integration Contracts

Step 1: Authentication

```

// Request
POST /api/v1/auth/login
Content-Type: application/json

{ "email": "agent@example.com", "password": "... " }

// Response — store access_token for all subsequent requests
{ "access_token": "eyJhbGciOiAi... ", "expires_in": 3600 }

```

Step 2: Upload Panorama Image (Multipart)

```

POST /api/v1/upload // or whichever file upload endpoint exists
Authorization: Bearer {access_token}
Content-Type: multipart/form-data

Body: file=@living_room_360.jpg

// Response
{ "file_url": "https://ethred-backend.onrender.com/uploads/living_room_360.jpg" }

```

Step 3: Attach Media to Property

```

POST /api/v1/properties/{property_id}/media // or POST /api/v1/properties (if inline)
Authorization: Bearer {access_token}
Content-Type: application/json

{
  "file_url": "https://ethred-backend.onrender.com/uploads/living_room_360.jpg",
  "media_category": "IMAGE",
  "sort_order": 1
}

```

Step 4: Upload Tour Manifest (Multi-Room)

```

POST /api/v1/properties/{property_id}/media
Authorization: Bearer {access_token}
Content-Type: application/json

{
  "file_url": "https://ethred-backend.onrender.com/uploads/tour_manifest_abc123.json",
  "media_category": "DOCUMENT",
  "sort_order": 0
}

```

4 Functional Requirements

4.1 Authentication & Authorization (FR-AUTH)

FR-AUTH-01

JWT Login via Existing Auth API

The application MUST authenticate users by sending credentials to `POST /api/v1/auth/login` and securely storing the returned JWT access token in Android `EncryptedSharedPreferences` (AES-256 encrypted).

- Token MUST be attached as `Authorization: Bearer {token}` header on every API request
- App MUST detect token expiry (HTTP 401 response) and trigger re-authentication flow without data loss
- Token MUST be cleared on explicit user logout

FR-AUTH-02

Role-Based Access Control

Only users with roles `AGENT` or `SELLER` (as returned in the JWT payload or user profile API) MUST be permitted to access the 360° capture feature. Buyers should see a read-only message directing them to the web viewer.

4.2 Gyroscope-Guided Frame Capture (FR-CAP)

FR-CAP-01

Hardware Sensor Initialization & Validation

On capture screen launch, the app MUST verify the presence of a hardware gyroscope using `PackageManager.FEATURE_SENSOR_GYROSCOPE`. If absent, a clear error dialog MUST be shown with the message: "This device does not have a gyroscope sensor, which is required for 360° capture. Please use a compatible Android device."

- Use `SensorManager.TYPE_ROTATION_VECTOR` for fused orientation (gyro + accelerometer + magnetometer)
- Sensor update interval: `SensorManager.SENSOR_DELAY_UI` (approx. 16ms)
- Apply low-pass filter to raw sensor data to smooth jitter:
$$\text{filteredAngle} = \alpha \times \text{previous} + (1 - \alpha) \times \text{raw}$$
 where $\alpha = 0.15$

FR-CAP-02**Spherical Target Grid (Capture Reticle Overlay)**

The capture UI MUST display a live augmented-reality style reticle overlaid on the CameraX viewfinder. The reticle maps the phone's current orientation to a 2D projection of the capture sphere, visually guiding the user to systematically cover all angles.

Capture Grid Structure:

- **Upper Ring (Pitch +30°):** 8 evenly spaced target dots at yaw intervals of 0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°
- **Horizontal Ring (Pitch 0°):** 12 target dots at yaw intervals of 30°
- **Lower Ring (Pitch -30°):** 8 evenly spaced target dots
- Total: 28 target dots (minimum; user may capture more)

Dot State Colors:

-  Gray: Not yet captured
-  Blue pulsing: Currently in alignment zone (within ±3°)
-  Green checkmark: Successfully captured

FR-CAP-03**Auto-Trigger Frame Capture**

The camera MUST automatically capture a frame using `CameraX ImageCapture.takePicture()` when the following conditions are met simultaneously:

- Phone orientation is within ±2° tolerance of a target dot
- Orientation has been stable within this zone for ≥ 300ms (debounce period)
- Camera auto-focus and auto-exposure have converged (AE state = CONVERGED)
- The target dot has NOT already been captured (prevents duplicate frames)

On capture trigger:

- Screen MUST flash white briefly (shutter animation)
- Device MUST produce haptic feedback (short single vibration)
- The captured dot MUST immediately turn green with a checkmark animation
- A progress indicator shows X/28 frames captured

FR-CAP-04**Capture Quality Settings**

The CameraX ImageCapture configuration MUST use the following settings for optimal stitching quality:

- Resolution: `ResolutionStrategy.HIGHEST_AVAILABLE_STRATEGY` (or minimum 12MP / 4032×3024)
- JPEG Quality: 95% (not 100% to avoid excessively large files)
- White Balance: `AWB_MODE_AUTO` — locked after first frame capture to ensure color consistency across all frames
- Exposure: AE locked after first frame capture
- Flash: Disabled (flash causes uneven exposure across panorama)
- Zoom: Fixed at 1.0× (no optical zoom)

FR-CAP-05**Manual Override & Completion**

The user **MUST** be able to manually tap any target dot to force capture at that position. The user **MUST** also be able to complete capture early once a minimum of 16 frames have been captured (covering approximately 80% of the horizontal rings) via a "Finish Capture" button.

FR-CAP-06**Session Recovery**

If the app is backgrounded or the device is rotated during capture, the session **MUST** resume from where it left off. Already-captured dots **MUST** remain green and the remaining frames must still be capturable. Captured frames **MUST** be persisted to the Room database immediately after capture.

4.3 On-Device OpenCV Stitching Engine (FR-STITCH)

FR-STITCH-01**Native C++ OpenCV NDK Integration**

The stitching engine **MUST** be implemented in C++ using OpenCV 4.8+ compiled for Android ARM64 (ABI: arm64-v8a and armeabi-v7a for compatibility). The native library **MUST** be exposed to Kotlin via a JNI bridge class.

- Native library file: `libstitcher.so`
- JNI class: `com.ethred.panorama.stitching.NativeStitcher`
- Method: `external fun stitchFrames(framePaths: Array<String>, gyroDegrees: FloatArray, outputPath: String): StitchResult`

FR-STITCH-02**AKAZE Feature Detection & Matching**

The stitching pipeline **MUST** use AKAZE (Accelerated-KAZE) feature detection as the primary algorithm. ORB **MUST** be available as a fallback for devices with insufficient RAM (<3GB) since AKAZE is more memory intensive.

Parameters for AKAZE:

- `descriptor_type = DESCRIPTOR_MLDB`
- `descriptor_size = 0` (full size)
- `threshold = 0.001f`
- Matching: `cv::FlannBasedMatcher` with LSH index (binary descriptors)
- Ratio test threshold: 0.75 (Lowe's ratio)
- RANSAC threshold: 4.0 pixels for homography rejection

FR-STITCH-03**Spherical Projection, Seam Carving & Multi-Band Blending**

The stitching pipeline MUST execute the full OpenCV detail stitching pipeline:

- Warper: `cv::detail::SphericalWarper` with focal length estimated from camera EXIF metadata (`focal_length_mm / sensor_width_mm × image_width_px`). Fallback: device model lookup table (provided in code as a hardcoded map of common devices).
- Exposure Compensation: `cv::detail::BlocksGainCompensator` with `block_size=32`
- Seam Finder: `cv::detail::GraphCutSeamFinder(COST_COLOR_GRAD)`
- Blender: `cv::detail::MultiBandBlender` with `num_bands=5`
- Output: `cv::Mat` converted to `cv::IMWRITE_JPEG_QUALITY=92`

FR-STITCH-04**Output Resolution & EXIF Metadata**

The stitching pipeline MUST produce an equirectangular JPEG at 2:1 aspect ratio:

Resolution tiers based on device RAM:

- >6GB RAM: 8192×4096 px (high resolution)
- 3–6GB RAM: 4096×2048 px (standard resolution)
- <3GB RAM: 2048×1024 px (compact resolution)

Required EXIF XMP metadata (GPano namespace):

- `GPano:ProjectionType = equirectangular`
- `GPano:UsePanoramaViewer = True`
- `GPano:FullPanoWidthPixels = [output_width]`
- `GPano:FullPanoHeightPixels = [output_height]`
- `GPano:CroppedAreaImageWidthPixels = [output_width]`
- `GPano:CroppedAreaImageHeightPixels = [output_height]`
- `GPano:CroppedAreaLeftPixels = 0`
- `GPano:CroppedAreaTopPixels = 0`

4.4 Gap Inpainting & Post-Processing (FR-FILL)

FR-FILL-01**Zenith (Ceiling/Sky) Inpainting**

The top circular region of the equirectangular output (the zenith, approximately 10–15% of image height) will be missing due to hardware limitations of handheld capture. The engine MUST apply `cv::inpaint(src, mask, dst, inpaintRadius=3, INPAINT_NS)` where the mask covers this missing region, propagating surrounding texture upward seamlessly.

FR-FILL-02**Nadir (Floor/Feet) Coverage**

The nadir (bottom circular region) will contain the user's feet and floor. The app **MUST** offer the user three nadir cap options before uploading:

- **Auto Inpaint:** Run `cv::inpaint` to fill the area with a continuation of the floor texture
- **Smooth Blur Gradient:** Apply a circular radial gradient feathering the bottom to neutral gray/black
- **Agency Logo Cap:** Overlay a pre-set circular agency/brand logo image (loaded from app settings) centered at the nadir point — this is the professional standard used by Matterport

FR-FILL-03**Automatic Horizon Leveling**

The stitching engine **MUST** auto-correct horizon tilt by analyzing the gravitational vector logged during capture sessions and applying a roll correction rotation to the final equirectangular output, ensuring the horizon appears straight when viewed in a browser.

4.5 Multi-Room Tour Builder (FR-TOUR)

FR-TOUR-01**Property Session Management**

The app **MUST** allow users to create a capture session linked to a specific property (identified by its backend `property_id`). Within each property session, users **MUST** be able to add, name, and re-order multiple individual room captures (up to 20 rooms per property).

FR-TOUR-02**Interactive Hotspot Placement Editor**

After stitching, the app **MUST** display an interactive 360° preview of each room (using an in-app `WebView` with `Pannellum.js` or `SceneView`). The user **MUST** be able to long-press anywhere on the 360° preview to drop a hotspot marker. A bottom sheet dialog **MUST** appear prompting them to select the target room this hotspot links to.

- Dropped hotspot position **MUST** record the Pannellum-compatible pitch and yaw coordinates (in degrees)
- Hotspots **MUST** be visually represented by an animated arrow icon in the preview
- Maximum 5 hotspots per room scene
- Hotspots **MUST** be editable (tap to change target) and deletable (long-press to remove)

FR-TOUR-03**Tour Manifest Generation**

When the user finalizes a multi-room tour, the app **MUST** generate a valid `tour_manifest.json` file (as defined in Section 3.3) and save it locally. This manifest **MUST** be uploaded to the backend as a separate file and registered as a `DOCUMENT` type media asset with `sort_order: 0`.

4.6 Backend Upload & Sync (FR-SYNC)

FR-SYNC-01**Network-Resilient Upload Queue**

All uploads (panorama JPEGs and manifest JSON) MUST be enqueued in the Room `upload_queue` table immediately. A WorkManager `UploadWorker` MUST process this queue with the following constraints:

- Constraint: `NetworkType.CONNECTED` (executes only when online)
- Retry policy: Exponential backoff starting at 30 seconds, max 5 retries
- If all retries fail, the queue item MUST be marked `FAILED` and the user MUST be shown an inline notification with a "Retry" action
- Upload progress MUST be shown in the Android notification drawer as a persistent notification with a progress bar

FR-SYNC-02**Thumbnail Generation**

Before upload, the app MUST generate a 512×256 px thumbnail JPEG of the equirectangular panorama for display in the property listing's media gallery. The thumbnail MUST be uploaded alongside the full-resolution panorama and registered as a separate IMAGE media entry.

FR-SYNC-03**Post-Upload Cleanup**

After all files for a property session have been successfully uploaded and confirmed by the backend (HTTP 200/201), the app MUST:

- Mark all queue items as `DONE` in the Room DB
- Delete raw captured frames from local storage (to free device storage)
- Retain only the final stitched JPEG locally (for offline preview)
- Display a success confirmation dialog with a "View on Website" deeplink option

4.7 Local Preview Viewer (FR-VIEW)

FR-VIEW-01**In-App 360° Preview Before Upload**

After stitching completes, the app MUST immediately display a fully interactive 360° panorama preview using a WebView-embedded Pannellum.js viewer (or SceneView) loaded from a local `file://` path — no internet connection required. The user MUST be able to pan, tilt, and zoom within the preview before deciding to approve or retake the panorama.

FR-VIEW-02**Capture Quality Scoring**

The app **MUST** display an automatic quality score on the preview screen (1–5 stars) derived from the stitching engine's internal metrics:

- Number of matched feature keypoints (higher = better)
- Reprojection error from RANSAC homography (lower = better)
- Percentage of inpainted regions (lower = better)
- A low quality score (≤ 2 stars) **MUST** trigger a prominent warning recommending retake

5 Non-Functional Requirements

5.1 Performance Requirements (NFR-PERF)

Metric	Target	Measurement Condition
Stitching Time (24 frames)	≤ 12 seconds	High-end device: Snapdragon 8 Gen 1+, 8GB RAM
Stitching Time (28 frames)	≤ 20 seconds	Mid-range device: Snapdragon 700-series, 6GB RAM
Stitching Time (28 frames)	≤ 45 seconds	Low-end device: MediaTek Helio G90, 4GB RAM
App Cold Start Time	≤ 2.5 seconds	Pixel 6a or equivalent
Camera Preview Latency	≤ 100ms	From app resume to live preview visible
Sensor-to-Reticle Lag	≤ 32ms	Gyro sensor update to reticle position update
Max RAM During Stitching	≤ 512MB	Monitored via Android Profiler during stitching worker
Upload Speed (LTE)	Full 4096×2048 JPEG ≤ 20 seconds	File size approx. 8–15MB, LTE ~50Mbps

5.2 Memory & Storage Management (NFR-MEM)

- Every `cv::Mat` object allocated in the native C++ layer MUST be explicitly released via `mat.release()` or destructor pattern to prevent native heap OOM crashes.
- Input frames MUST be processed one-at-a-time in the feature detection phase, not all loaded simultaneously into memory.
- App MUST check available storage before starting a capture session. Minimum required: 500MB free (for 36 raw frames at ~5MB each + output panorama + temp files). If insufficient, the user MUST be warned with an option to free storage before proceeding.
- Raw frames MUST be deleted from local storage immediately after successful upload to the backend.
- Stitched panoramas MUST be kept locally for 30 days then automatically removed if they have been successfully uploaded (configurable in settings).

5.3 Compatibility Requirements (NFR-COMPAT)

Requirement	Value
Minimum SDK	Android 7.0 (API 24)
Target SDK	Android 15 (API 35)
Native ABI Targets	arm64-v8a (primary), armeabi-v7a (fallback)
Required Hardware Sensor	Gyroscope (<code>FEATURE_SENSOR_GYROSCOPE</code>)
Required Minimum RAM	3GB physical RAM
Required Minimum Camera	8MP rear camera with autofocus
Display Resolution	Minimum 1080×1920 px
Internet	Required only for upload; capture + stitching = fully offline

5.4 Reliability & Offline Resilience (NFR-RELY)

- The app MUST be functional offline for all stages except upload (capture, stitch, preview, hotspot editing).
- WorkManager upload tasks MUST survive device reboots — they MUST be re-enqueued via `EXPEDITED` work type with persistent constraints.
- If the stitching process is killed mid-way (due to OOM or backgrounding), the app MUST NOT delete any captured raw frames. The user MUST be prompted to retry stitching when they reopen the session.
- Crash rate target: < 0.1% of sessions (measured via Firebase Crashlytics).
- ANR rate target: < 0.05% of sessions.

5.5 Usability & Accessibility (NFR-UX)

- All text MUST have minimum contrast ratio of 4.5:1 against background (WCAG 2.1 AA).
- All interactive elements MUST have a minimum touch target size of 48×48 dp.
- The guided capture flow MUST be completable by a first-time user without reading any external documentation — onboarding MUST include a 30-second interactive tutorial on first launch.
- The app MUST support landscape and portrait orientation in the capture view.
- All haptic feedback MUST be suppressable in Settings for users with sensory sensitivities.
- App MUST support TalkBack accessibility (all UI elements MUST have meaningful `contentDescription` attributes).

5.6 Localization (NFR-L10N)

- The app MUST support English (en) and Amharic (am) languages as initial languages.
- All UI strings MUST be externalized to `strings.xml` (no hardcoded strings in Kotlin/Compose code).
- Date and time formatting MUST respect device locale settings.

6 Use Case Scenarios

UC-01: Agent Captures First 360° Room Panorama

UC-01: First Time 360° Capture — Single Room

Actor: Real Estate Agent | **Precondition:** Logged in, existing property in backend system, phone has hardware gyroscope

- 1 **Agent Action:** Opens the Ethred app and navigates to their property dashboard. Selects the property "Villa #307, Bole" and taps the "+ Add 360° Photos" button.
- 2 **System Action:** Displays the Room Setup screen. Agent types room name "Living Room" and taps "Start Capture".
- 3 **System Action:** Opens CameraX viewfinder with the spherical reticle overlay. 28 gray target dots appear floating over the camera feed. A voice/text prompt reads "Stand in the center of the room. Hold your phone upright. Slowly rotate 360°."
- 4 **Agent Action:** Slowly rotates in place. As phone orientation approaches target dots, dots pulse blue.
- 5 **System Action:** On alignment lock: *click* sound + haptic. Dot turns green with checkmark. Frame saved to session. Counter updates: "5/28".
- 6 **Agent Action:** Completes all 28 dots after about 3–4 minutes. Taps "Finish Capture".
- 7 **System Action:** Stitching progress screen shown. Animated progress indicator with stage labels: "Detecting features... Warping... Blending... Filling gaps... Done!". Completes in ~12 seconds on the agent's Pixel 7a.
- 8 **System Action:** Full interactive 360° preview opens in-app. Quality score: ★★☆☆ (4/5). Nadir cap selector shows. Agent selects "Agency Logo".
- 9 **Agent Action:** Reviews panorama, satisfied. Taps "Upload to Property Listing".
- 10 **System Action:** WorkManager queues upload. JPEG uploaded to backend (POST /upload). media_category: IMAGE attached to Property #307 (POST /api/v1/properties/{id}/media). Notification shows upload progress.
- 11 **System Action:** Upload complete. Toast: "✅ 360° photo published to Villa #307 listing." Property #307's web listing now shows an interactive 360° viewer powered by Pannellum.js.

UC-02: Agent Creates Multi-Room Virtual Tour

UC-02: Multi-Room Tour With Hotspot Navigation

Actor: Real Estate Agent | **Precondition:** UC-01 completed for Living Room; agent wants to add Kitchen and link them

- 1 **Agent Action:** On property session screen, taps "+ Add Another Room". Names it "Kitchen".
- 2 **System Action:** Opens capture screen again. Agent walks to kitchen and captures 28 frames. Stitching completes.
- 3 **Agent Action:** On Tour Editor screen, opens Living Room 360° preview. Long-presses near the kitchen doorway in the panorama.
- 4 **System Action:** Bottom sheet opens: "Add Hotspot — Where does this take you?" Lists: Living Room ✓ · Kitchen · Bedroom. Agent selects "Kitchen".
- 5 **System Action:** Blue arrow hotspot icon appears at the doorway position. Pitch: -4.8°, Yaw: 113.2° recorded. Agent repeats for Kitchen → Living Room link.
- 6 **Agent Action:** Taps "Publish Tour".
- 7 **System Action:** Generates `tour_manifest.json`. Uploads both panorama JPEGs (IMAGE type) and the manifest file (DOCUMENT type). All 3 files attached to Property #307.
- 8 **Result:** Property #307 web page shows a full multi-room virtual tour. Buyers can click the kitchen doorway arrow in the Living Room to seamlessly transition to the Kitchen view — exactly like Redfin/Matterport.

UC-03: Offline Capture With Delayed Upload

UC-03: Offline Capture → Auto-Sync When Connected

Actor: Agent in a basement/rural area without network signal | **Precondition:** Phone in airplane mode or no signal

- 1 **Agent Action:** Opens app in offline mode. Property data previously loaded is available from Room DB cache.
- 2 **Agent Action:** Captures and stitches 3 rooms fully offline. All data persisted in Room DB and local file system.
- 3 **System Action:** Upload queue shows "3 rooms pending upload". All items in PENDING status.
- 4 **System Action:** Agent drives back to office, phone connects to Wi-Fi. WorkManager NetworkType.CONNECTED constraint triggers automatically.
- 5 **System Action:** Background UploadWorker runs. All 3 rooms + manifests uploaded sequentially. Persistent notification shows "Uploading 360° photos — 2/3 complete".
- 6 **System Action:** Upload complete. Push notification delivered: "✅ Sunshine Apartment — 3 rooms published successfully." Raw frames deleted from local storage.

UC-04: Low Quality Stitching — Retake Prompt

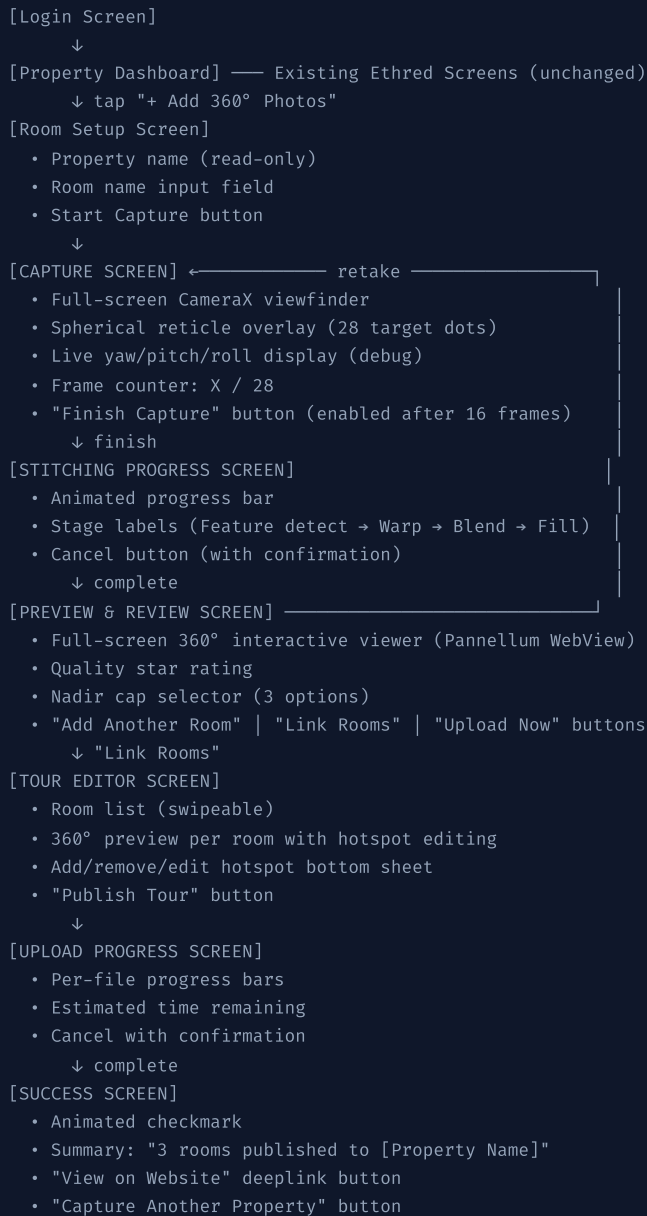
⚠️ UC-04: Stitching Fails Quality Check → Agent Retakes

Precondition: Agent captured frames too quickly, insufficient overlap between frames

- 1 **System Action:** Stitching completes but quality score = ★ (1/5 stars). RANSAC inlier ratio: 0.23 (below 0.5 threshold). Significant visible seam artifacts detected.
- 2 **System Action:** Warning card displayed: "⚠️ Low Quality Panorama — Feature matching was poor, likely due to fast movement or insufficient overlap. Visible seams may appear. We strongly recommend retaking this room." Buttons: "Retake" (primary) | "Upload Anyway" (secondary).
- 3 **Agent Action:** Taps "Retake". App discards current stitch but retains captured raw frames. Agent is taken back to the capture screen to re-capture additional frames to improve coverage.
- 4 **System Action:** Second stitch with improved frames. Quality score: ★★★★★ (4/5). Agent approves and uploads.

7 Screen & UI/UX Specification

7.1 Screen Flow Diagram



7.2 Design System Tokens

Token	Value	Usage
Primary Blue	#1A56DB	Buttons, active states, accent elements
Capture Blue	#3B82F6	Pulsing reticle dot (alignment zone)
Success Green	#16A34A	Captured dots, success messages
Warning Amber	#D97706	Low quality warnings
Error Red	#DC2626	Error states, upload failures
Background	#0F172A	Capture screen dark overlay
Surface	#1E293B	Cards, bottom sheets
Typography	Inter / Roboto	All text elements
Border Radius	12dp (cards), 8dp (buttons)	Rounded elements

7.3 Onboarding First-Launch Tutorial

On first app launch after install, an interactive tutorial MUST walk the user through:

- Step 1 — How the reticle works:** Animated GIF/video showing the dots and how they turn green
- Step 2 — Correct body positioning:** Stand in the center of the room, keep phone vertical
- Step 3 — Movement speed:** Rotate slowly and steadily — app pauses animation to show "too fast" vs "correct speed"
- Step 4 — Lighting tips:** Open curtains, turn on all lights for best results
- Step 5 — "You're ready":** Tap "Start Capturing" to go to the property selection screen

Tutorial MUST be skippable and re-accessible from the Settings menu at any time.

8 Error Handling & Edge Cases

Error Scenario	Detection Method	User-Facing Message	System Action
No hardware gyroscope	<code>PackageManager.hasSystemFeature(FEATURE_SENSOR_GYROSCOPE) = false</code>	"Your device doesn't have a gyroscope. 360° capture is not available."	Disable capture button entirely. Show alternative (manual upload option).
Camera permission denied	CameraX returns <code>CameraState.ERROR_CAMERA_IN_USE</code> or permission not granted	"Camera access is required. Please grant camera permission in Settings."	Show Permission Rationale dialog with deep link to App Settings.
Stitching fails (too few features)	OpenCV sticher returns <code>Stitcher::ERR_NEED_MORE_IMGS</code> or feature match ratio < 0.15	"Not enough overlap between photos. Please retake with slower rotation."	Discard stitching result. Retain raw frames. Offer retake.
Native library crash (OOM)	JNI returns error code -1 or app receives SIGKILL	"Processing ran out of memory. We'll try a lower resolution."	Retry stitching at next lower resolution tier (fallback chain: 8192→4096→2048).
Insufficient storage	Available storage < 500MB before capture starts	"Not enough storage. Please free at least 500MB and try again."	Block capture start. Show storage usage breakdown.
Upload authentication failure (401)	Backend returns HTTP 401 on upload request	"Session expired. Please log in again."	Pause upload queue. Open login screen. Resume queue after re-authentication.
Upload network error (<5 retries)	OkHttp throws IOException / backend returns 5xx	Silent retry (no user interruption)	Exponential backoff: 30s, 60s, 2m, 5m, 10m. Notification on final failure.

Error Scenario	Detection Method	User-Facing Message	System Action
Upload network error (5+ retries)	WorkManager marks task FAILED after max retries	"Upload failed for [Room Name]. Tap to retry."	Show persistent notification with "Retry" action. Mark queue item as FAILED in Room DB.
Phone rotation during stitching	Activity configuration change	No interruption — stitching progress shown	StitchWorker is decoupled from Activity. Reconnect via WorkManager observation on resume.
Backend property not found (404)	Backend returns HTTP 404 on media attach request	"Property not found on server. It may have been deleted."	Mark session as orphaned in Room DB. Offer to create new property.

9 Security Requirements

SEC-01

Secure Token Storage

The JWT access token MUST be stored in `EncryptedSharedPreferences` using `MasterKey.Builder` with AES-256-GCM encryption backed by Android Keystore. The token MUST NEVER be stored in plain `SharedPreferences`, plain text files, or the Room database.

SEC-02

Certificate Pinning

All HTTPS network calls to the Ethred backend MUST use OkHttp's `CertificatePinner` with the backend server's SSL certificate SHA-256 hash pinned. This prevents man-in-the-middle (MITM) attacks on the uploaded photo data.

SEC-03

No Cloud Key Management for Free Services

As all processing is on-device and free, no API keys, external AI/cloud service keys, or third-party service credentials are stored or transmitted. The only credentials handled are user JWT tokens from the existing Ethred auth service.

SEC-04

Local File Privacy

Captured frames and stitched panoramas MUST be stored in the app's internal storage directory (`context.filesDir`), which is not accessible to other apps or the Android media gallery without explicit user share action. The `android:sharedUserId` attribute MUST NOT be used.

SEC-05

ProGuard / R8 Obfuscation

Release builds MUST enable R8 code shrinking and obfuscation to protect business logic. Native C++ code is inherently binary-only. ProGuard rules MUST be configured to preserve Room DB, Retrofit interfaces, and Hilt DI entry points.

10 Testing Strategy

10.1 Unit Tests (JUnit5 + MockK)

Component	Test Cases
SensorOrientationProcessor	Low-pass filter correctness; alignment tolerance calculation; yaw wrap-around at 359°→0°
CaptureGridManager	Correct generation of 28 target dot positions; duplicate detection logic
ManifestGenerator	Valid JSON output for 1/2/5 rooms; hotspot serialization; missing rooms graceful handling
UploadQueueRepository	Queue insertion; retry count increment; status transitions; cleanup after success
AuthInterceptor	Token injection; 401 handling; token refresh flow

10.2 Instrumentation Tests (Espresso / UIAutomator)

- End-to-end capture flow using a mock sensor feed (inject simulated rotation events via `SensorManager.injectSensorData()`)
- WorkManager upload test using `TestWorkManagerFactory` to validate queue processing logic
- Room DB migration tests for schema version upgrades
- WebView Pannellum.js preview load test on real device

10.3 Native C++ Unit Tests (Google Test Framework)

- Feature detection returns non-empty keypoints for known test images
- Spherical warp produces correct output dimensions for given focal length
- Inpainting masks produce visually coherent fills on synthetic test images
- EXIF XMP metadata is correctly embedded in output JPEG
- Memory leak detection run via Valgrind/ASan on x86_64 emulator

10.4 Device Compatibility Matrix

Device	Chip	RAM	Android	Test Status
Samsung Galaxy A54	Exynos 1380	8GB	Android 13	REQUIRED
Xiaomi Redmi Note 12	Snapdragon 4 Gen 1	4GB	Android 12	REQUIRED
Tecno Spark 10	MediaTek Helio G88	4GB	Android 12	REQUIRED
Google Pixel 7a	Google Tensor G2	8GB	Android 14	REQUIRED
Samsung Galaxy S23	Snapdragon 8 Gen 2	8GB	Android 13	OPTIONAL

Note on Ethiopian Device Market. The primary target users are Ethiopian real estate agents. Common devices in the Addis Ababa market include Tecno, Infinix, and Samsung mid-range devices. The Tecno Spark 10 and

Samsung Galaxy A54 MUST be prioritized for testing as representative low-to-mid range Ethiopian market devices. These typically have 4–8GB RAM and Helio G88 / Exynos 1380 chips.

11 Developer Implementation Prompt

Use the following prompt verbatim when delegating development of this Android application to an AI coding assistant (such as Claude, GPT-4o, or Gemini) or when onboarding a new Android developer:

You are an expert Android developer specializing in Kotlin, Jetpack Compose, CameraX, and native C++ (NDK/JNI) development. Build a production-ready Android application called "Ethred 360° Capture" with minSdk 24, targetSdk 35.

PURPOSE

Enable real estate agents to:

1. Capture 360° equirectangular panoramas using their phone camera, guided by gyroscope sensor AR overlays
2. Stitch photos on-device using native OpenCV 4.8 (C++ via Android NDK)
3. Fill ceiling (zenith) and floor (nadir) gaps using cv::inpaint
4. Link multiple rooms into a virtual tour with interactive hotspots
5. Upload finished panoramas to the Ethred Backend API (Node.js/Prisma) WITHOUT modifying the existing backend DB schema

CAPTURE SYSTEM

- Use Android SensorManager (TYPE_ROTATION_VECTOR) for 3-axis orientation
- Display a spherical reticle overlay on the CameraX viewfinder with 28 target dots: 8 at +30° pitch, 12 at 0° pitch, 8 at -30° pitch
- Auto-capture via CameraX ImageCapture when orientation is within ±2° of a target dot for 300ms with AE state CONVERGED
- Lock White Balance and Exposure after first frame to ensure consistency
- Haptic + shutter animation on each frame capture
- Progress counter: X/28 frames captured
- Allow early finish at 16+ frames with user confirmation

STITCHING PIPELINE (Native C++ / OpenCV NDK)

JNI Bridge class: com.ethred.panorama.stitching.NativeStitcher
Native library: libstitcher.so (arm64-v8a + armeabi-v7a)

Pipeline steps:

1. Decode frames to cv::Mat, downsample to 1920×1440 for feature detection
2. AKAZE feature detection (fallback: ORB for <3GB RAM devices)
3. FLANN-based matching with Lowe's ratio test 0.75
4. RANSAC homography estimation (threshold: 4.0 px)
5. cv::detail::SphericalWarper (focal length from EXIF or device table)
6. cv::detail::BlocksGainCompensator (block_size=32)
7. cv::detail::GraphCutSeamFinder (COST_COLOR_GRAD)
8. cv::detail::MultiBandBlender (num_bands=5)
9. Output at 4096×2048 (mid-range) or 8192×4096 (high-end, >6GB RAM)
10. cv::inpaint zenith + nadir missing regions (INPAINT_NS method)
11. Optional: Nadir logo cap overlay from agency branding settings
12. Embed GPano XMP EXIF metadata into output JPEG
13. Generate 512×256 thumbnail for property gallery

Quality score (1-5 stars) based on:

- RANSAC inlier ratio (higher = better)
- % inpainted area (lower = better)
- Feature match count (higher = better)

If quality score ≤ 2: show prominent retake warning

MULTI-ROOM TOUR BUILDER

- Users can capture up to 20 rooms per property session
- After stitching, show interactive 360° preview via Pannellum.js in a WebView loading from local file:// path
- Long-press on 360° preview → drop hotspot → select target room → record pitch/yaw coordinates
- Generate tour_manifest.json (see schema below) on "Publish Tour"

- Upload manifest as DOCUMENT type media to backend

tour_manifest.json schema:

```
{
  "version": "1.0",
  "property_id": "uuid",
  "default_scene": "scene_id_string",
  "scenes": [
    {
      "id": "scene_living_room",
      "name": "Living Room",
      "file_url": "https://backend.com/uploads/living_room_360.jpg",
      "thumbnail_url": "https://backend.com/uploads/living_room_thumb.jpg",
      "hotspots": [
        {
          "id": "hs_uuid",
          "pitch": -5.2,
          "yaw": 112.5,
          "text": "Go to Kitchen",
          "target_scene_id": "scene_kitchen"
        }
      ]
    }
  ]
}
```

BACKEND API INTEGRATION (ethred-backend – NO SCHEMA CHANGES)

Base URL: <https://ethred-backend.onrender.com/api/v1>

1. Auth: POST /auth/login → { email, password } → store JWT in EncryptedSharedPreferences
2. Upload file: POST /upload (multipart/form-data, field: "file") → { file_url }
3. Attach media to property: POST /properties/{id}/media → { file_url, media_category: "IMAGE"|"DOCUMENT", sort_order: int }

WorkManager UploadWorker:

- Constraint: NetworkType.CONNECTED
- Retry: exponential backoff (30s, 1m, 2m, 5m, 10m max)
- Show persistent notification with progress bar
- Delete raw frames after successful upload
- Keep stitched panorama locally for offline preview

Existing Prisma model used (no changes):

```
model PropertyMedia {
  id          String    @id @default(uuid())
  property_id String
  file_url    String
  media_category MediaType // IMAGE | VIDEO | DOCUMENT
  sort_order  Int
}
```

LOCAL ROOM DATABASE SCHEMA

Tables: capture_sessions, captured_frames, hotspots, upload_queue
(Full schema documented in SRS Section 3.2)

ARCHITECTURE

Pattern: MVVM + Clean Architecture

- presentation layer: Jetpack Compose + ViewModels
- domain layer: UseCases (pure Kotlin, no Android deps)
- data layer: Repositories (Room DB + Retrofit API)
- native layer: C++ OpenCV stitcher via JNI

DI: Hilt
Async: Kotlin Coroutines + StateFlow
Image loading: Coil

COMPATIBILITY

- minSdk: 24 (Android 7.0)
- targetSdk: 35 (Android 15)
- ABI: arm64-v8a (primary), armeabi-v7a (fallback)
- Required: Hardware gyroscope (FEATURE_SENSOR_GYROSCOPE)
- Min RAM: 3GB
- Languages: English (en), Amharic (am)

UI DESIGN

- Material Design 3, dark theme for capture screens
- Capture reticle: dark overlay with glowing target dots (gray=uncaptured, blue pulsing=in zone, green check=captured)
- First-launch onboarding tutorial (5 steps, skippable)
- All text in strings.xml (no hardcoded strings)

TESTING

- Unit tests: JUnit5 + MockK for all UseCases and Repositories
- Instrumentation: Espresso for UI flows
- Native: Google Test for C++ stitching pipeline
- Memory leak detection: LeakCanary
- Crash tracking: Firebase Crashlytics (free tier)